

Predicting the Future of Computer Architectures

Introduction

Computer architecture is the design and organisation of components which make up a computer system. There are several components that can construct a basic architecture which include the CPU (central processing unit), Memory (RAM, registers and cache memory) and the Instruction set architecture (ISA) etc.

The CPU is the brains of the computer; this system is on a physical chip called the microprocessor. The microprocessor is responsible for computing calculations and executing instructions. It does arithmetic using its Arithmetic Logic Unit (ALU) and fetches data from registers and its cache memory. By developing and optimising processors, it should be able to see that it leads to exponential growth in computation.

Brief history of the microprocessor

Designed by Charles Babbage, the analytical engine was developing between 1834-1871. As a successor to the difference engine, which is described in his autobiography [1], this was meant to be the first mechanical calculator. It had gears with 10 inputs (0-9) and was quite a large machine as seen in figure 1.

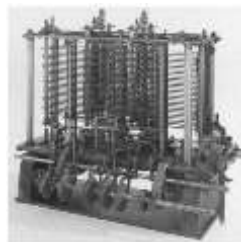


Figure 1

The vacuum tube was invented in 1907 and was basically a switch controlled by releasing electrons by heating up a cathode via thermionic emission. These were too large since made from glass it would break easily [2].

1947 was the first instance of a working point contact transistor. This was revolutionary as they were much faster and smaller than the vacuum tubes. This also allowed designs of logic gates and eventually with the invention of Bipolar Junction Transistors (BJT's) in 1948 by William Shockley and his team at the Bell labs. They were more stable and led to integrated circuits (IC, also known as microchips) in 1975. A microprocessor is a type of microchip designed for computing.

Trends of microprocessors and Moore's law

Since the invention of the microchip, we have been fitting more transistors per microchip [3]. Moore's law states that the number of transistors per IC doubles every two years roughly and we have seen this trend occur in figure 2.

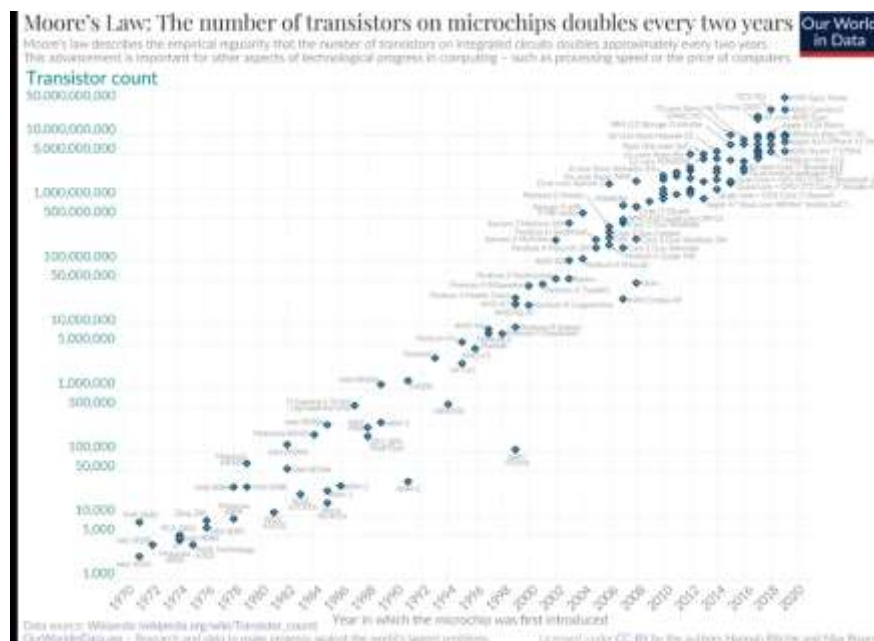
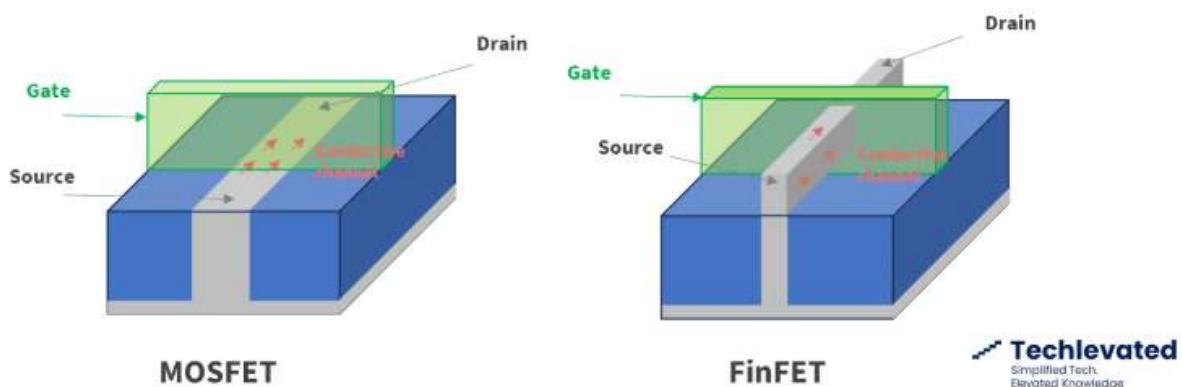


Figure 2

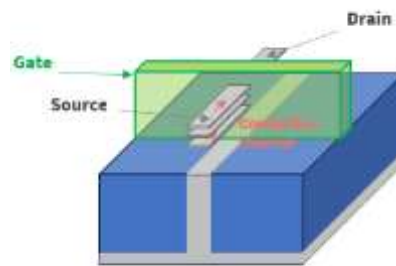
By increasing the number of transistors per microchip, we are increasing the computing performance of our systems [4]. With more transistors comes more CPU cores which means that we can allow more parallel processing hence more tasks can be executed at the same time [5]. Following this trend leads to challenges such as physical limits, economic limits and technical challenges. As we continue to develop the technologies, we spend more money for less increase in performance at a certain point [6] (>180nm) we would find challenges as when we decreased the gate length, we would find that there would be current leakage (electron tunnelling) due to the channel being too thin (We can't control the transistor). This led to the innovation of the FinFET where we would wrap the gate around the channel in a 3D space (figure 3) [7]. This vertical fin allows the same effect as the MOSFET but in a more tightly packed area due to this 3D architecture



allowing better gate control since more contact area and lessening short channel effects. This was commercialised in the 2010's.

Figure 3

Eventually this limit was also reached leading to GAAFET (figure 4) which would have the gate fully surrounding the channel (now nanosheets) allowing us to scale up the density of transistors even more. First proposed in 1999, it took decades to get the practical application which we achieved in the 2020's



GAAFET

Figure 4

Short term future of microprocessors

There is a new architecture in development called a CFET which is a vertical transistor (figure 5). Here the nMOS and pMOS are vertical rather than side by side and use nMOS and pMOS nanosheets. There are many challenges with this such as interconnecting the transistors together and also heat dissipation. More heat produced with less area per transistor combined with more transistors will lead to performance limitations as there are less ways to dissipate this heat leading to more resistances.

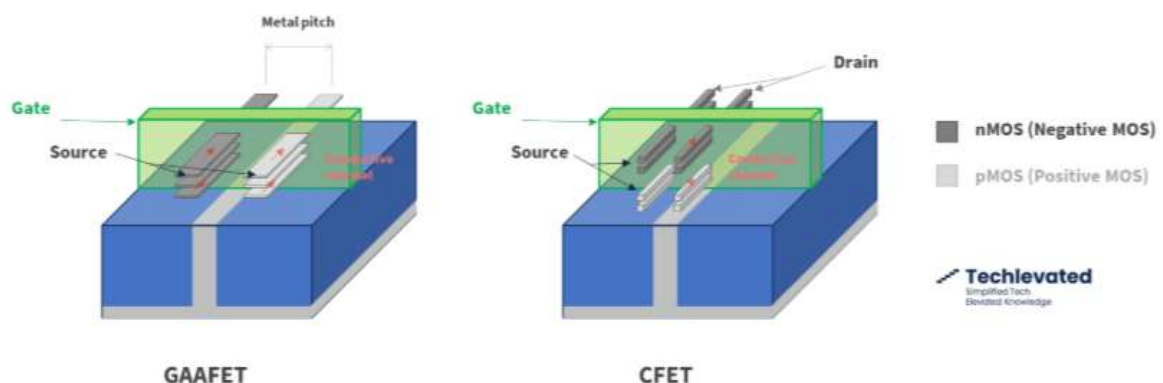


Figure 5

Architectural needs and long term future predictions

There has always been a need for greater computing power resulting in development of better architectures from making parallelism in GPU/CPU cores to AI. Warehouses are filled with thousands of CPU's, GPU's and servers all dedicated to machine learning and training ai which has led to greater need for microprocessor innovation as the costs to

train the AI models skyrocket. Increasing the efficiency of these chips will save millions of pounds for these companies so future development is extremely important.

There is a physical limit in silicon as we can only reduce the gate width by so much before leakage occurs and, since we are approaching 1nm transistors [8], we are approaching transistors at atomic sizes. “It is becoming more difficult to scale Silicon CMOS further without sacrificing performance, which is driving research into other channel technologies materials” [9]. We may need to find other materials such as 2-d materials (1 atom layer) which have better heat dissipation and ultrathin bodies so that we can scale transistor sizes down even more without running into leakage issues. Eventually there will be a physical limit to these materials so Moore’s law will fail, not to mention that the performance increases at less amounts per transistor count therefore not as much economic incentive. 2-D materials can lead to problems such as quantum tunnelling which will increase in probability with thinner materials. This tunnelling is essential to physics so can’t exactly be stopped but in the future, there may be ways to use this efficiently with better architectural design.

“And yet, by imposing appropriate boundary conditions on this function we were able to determine the probability that a particle (represented by a *localized* wave packet) would bounce off, or pass through, the potential” [9] shows that it is possible to use probability to calculate how much quantum tunnelling will affect the microprocessor.

To be a bit more speculative, we already have a “CPU” which is the brain. It far exceeds any microprocessor we have in complexity. It uses little to no power, with no heat dissipation requirements. It is a dynamic system as it adapts from past events. We can see in modern times that with the rise of language models, there is an incentive to mimic human interaction and machine learning, just like the brain, is all about adapting and learning from experience. “Artificial Intelligence, through its working horse of neural networks, is inspired by the biological example of the brain” [10].

With the use of supercomputer doing intense simulation, it may be found out that it is the best microprocessor simply because it has had millions of years to develop. It is literally designed perfectly for its environment. If we ever do manage to develop this technology in the far future, it will surely have moral challenges (is the synthetic brain also a life?) and philosophical questions on the sentience of humanity.

Task 2A

Factorisation by recursion (C):

```
#include <stdio.h>
```

```

int test_input[] = {4, 3, 7, 8};           //data test_input: .word 4 3 7 8.
int *t2= test_input;                     //la t2, test_input, We can set the base address to temporary register t2.

int factorial(int n) {                    //factorial: n is the value in register a0.
                                          //addi sp, sp, -8, allocate stack space.
                                          //sw ra, 4(sp), Write return address into memory.
                                          //sw a0, 0(sp), Save n for after recursion.

    if (n <= 1) {                        //This is the base case.
                                          //addi t0, x0, 1, will be needed for comparison.
                                          //bge t0, a0, base_return, When n<=1 jump to base_return label.

        return 1;
    } else {                             //When we need recursion to factorise.
                                          //addi a0, a0, -1 (n = n-1), we are decreasing n for each recursion.
                                          //jal ra, factorial, call upon factorial function.
                                          //lw a1, 0(sp), load original n into a1.
                                          //addi t0, x0, 0, t0 = 0 for computation of multiplication.
                                          // addi t1, x0, 0 // Initialize loop counter (t1 = 0).

                                          //multiplication:
                                          //beq t1, a1, end_multiply , Exit when counter equals n.
                                          //add t0, t0, a0 / Add factorial(n-1) to accumulator.
                                          //addi t1, t1, 1 Increment counter ++.
                                          //jal x0,multiplication .
                                          //end_multiply:
                                          //add a0, x0, t0, this is a0 final result.
                                          //lw ra, 4(sp), restoring the return address.
                                          //addi sp, sp, 8, deallocate the stack space .
                                          //jr ra, for returning to ra address.

        return n * factorial(n - 1);
    }
}

int main(){
    int t1 = 4;                          //addi t1, x0, 4, Setting the temporary register 1 equal to array length.
    int t0=0;                            //add t0, x0, x0, Making the temporary register equal 0. This means the number.
                                          //represents how far along the array we are(usually i in C).

    while(t0!=t1){                       //beq t0, t1, main_exit, Once this condition is satisfied we exit the loop.
        int a0= test_input[t0];

                                          //slli t3, t0, 2, Calculating address of array element by *2^2 since words are 4
                                          //bytes. Set this to a new temporary register t3.
                                          //add t4, t2, t3, We then add base address + offset for new address and set it to
                                          //temporary register t4.
                                          //lw a0, 0(t4), a0 now contains the value at address t4.
                                          //addi sp, sp, -20 [sw t0, 0(sp) sw t1, 4(sp) sw t2, 8(sp) sw t3, 12(sp) sw t4, 16(sp)], caller
                                          //saved.
                                          //5 Registers to be stored in multiples of 4 since registers are 4 bytes long.
    }
}

```

```

int t3 = factorial(a0);    //jal ra, factorial. After function ends it returns to ra whilst the return value placed
                           //in a0.
                           //lw t0, 0(sp).
                           //lw t1, 4(sp).
                           //lw t2, 8(sp).
                           //lw t3, 12(sp).
                           //lw t4, 16(sp).
                           //addi sp, sp, 20.

printf("%d ", t3);        //addi a1, t3, 0, This is moving t3 value to the argument register a1.
                           //addi a0, x0, 1, This is the call code for printing an integer.
                           //ecall, Execute the call code.
                           //addi a1, x0, ' '; The space character ASCII is now in a1.
                           //addi a0, x0, 11, Call code for printing a character.
                           //ecall, Execute the call code.
t0++;                     //addi t0, t0, 1, Add +1 to the t0 (i in C usually).
                           //jal x0, main_loop, Start the loop again.
}
exit(0);                  //addi a0, x0, 10, 10 is ecall code to exit.
                           //ecall #exit.
}

```

This method has a function that calls upon itself until it reaches the lowest factorial which is 1. It then starts moving upwards and multiplies (by addition) going up the 'stack ladder'.

Factorisation by iteration (C)

```

#include <stdio.h>          //Comments same as previous except for the different factorial function.

int test_input[] = {4, 3, 7, 8};
int *t2 = test_input;

int factorial(int n) {      //Iterative function instead of recursion.
    int result = 1;         //addi t0, x0, equals 1 so because that's the lowest factorial.
    for (int i = 1; i <= n; i++) { //So i(t0) is the loop counter, when this equals the array length n then exit
        result *= i;        //result is * by i every iteration of the loop.
    }
    return result;
}

int main() {
    int t1 = 4;
    int t0 = 0;

    while (t0 != t1) {
        int a0 = test_input[t0];
        int t3 = factorial(a0);
        printf("%d ", t3);
        t0++;
    }

    return 0;
}

```

This factorises by multiplying from $1*2 + 2*3 + 3*4...$ etc. It is different from recursion because it has a set loop and does not need to check itself after each loop for when to break the loop.

Factorisation by recursion (RISCV):

```
.data
test_input: .word 4 3 7 8           #data to be factorised

.text
main:
    add t0, x0, x0                  #t0 = 0, it represents loop counter [i].
    addi t1, x0, 4                  #t1 = 4, Number of elements in array setting the loop limit.
    la t2, test_input               #t2 address is pointing to the address at test_input.
main_loop:
    beq t0, t1, main_exit           #When t0 == t1, leave the loop (After the final element, exit the loop).
    slli t3, t0, 2                  #Multiplying t0 by 4 since we want to get the byte offset. slli is multiplication
                                    #by 2^n
    add t4, t2, t3                  #t4 = base address + base offset. Address of test_input [i].
    lw a0, 0(t4)                   #Load test_input[i] into a0 argument register (to be inputted to the factorial
                                    #function).

    addi sp, sp, -20                #Allocate stack space (-20 for 5 registers (5*4)).
    sw t0, 0(sp)                   #Write t0(loop counter) into memory.
    sw t1, 4(sp)                   #Write t1(loop limit) into memory.
    sw t2, 8(sp)                   #Write t2(pointer to base address of array) into memory.
    sw t3, 12(sp)                  #Write t3(byte offset) into memory.
    sw t4, 16(sp)                  #Write t4(Address of test_input [i]) into memory.

    jal ra, factorial              #Call the factorial function and return address to be in ra.

    lw t0, 0(sp)                   #Restoring all the registers from stack.
    lw t1, 4(sp)
    lw t2, 8(sp)
    lw t3, 12(sp)
    lw t4, 16(sp)
    addi sp, sp, 20                #Deallocate stack space.

    addi a1, a0, 0                 #Move a0 to a1 to print.
    addi a0, x0, 1                 #a0 = 1 for ecall code to print integer.
    ecall # Print Result           #Execute the instruction.
    addi a1, x0, ' '               #a1 = ASCII for space character.
    addi a0, x0, 11                #a0 = 11 for ecall code to print character.
    ecall                          #Execute code for print character.

    addi t0, t0, 1                 #t0++, increment loop counter.
    jal x0, main_loop              #Jump to the main_loop.
main_exit:
    addi a0, x0, 10                #a0 = 10, Ecall code for exit.
    ecall # Exit                   #Execute instruction to exit programme.

factorial:
```

addi t0, x0, 1	#t0 = 1, used for comparison.
bge t0, a0, base	#If a0(n) <= 1 then branch to base label. Basically, when n = 1 or 0.
addi sp, sp, -8	#Stack space allocated for two registers (The return address and current n value)
sw ra, 4(sp)	#Write ra(return address) into memory.
sw a0, 0(sp)	#Write a0(n) into memory.
addi a0, a0, -1	#a0 = a0 - 1, so a0 = (n-1)
jal ra, factorial	#Jumps to factorial and begin loop again. We keep looping until we get to 1.
lw t0, 0(sp)	#Load original n value to t0.
addi t2, a0, 0	#t2 = a0. t2 holds the value (n-1)!
addi a0, x0, 0	#a0 = 0, a0 to be used for multiplication.
 multiplication:	
beq t0, x0, multiplication_exit	#If t0 == 0 then exit loop.
add a0, a0, t2	#a0 = a0 + t2.
addi t0, t0, -1	#Loop counter goes down by one.
jal x0, multiplication	#Jumps back to the multiplication loop until exit condition satisfied.
 multiplication_exit:	
lw ra, 4(sp)	#Load the next n factorial.
addi sp, sp, 8	#Deallocate stack space.
jr ra	#Return to caller
 base:	
addi a0, x0, 1	#a0 = 1, since 1! and 0! = 1 so return 1.
jr ra	#Return to caller

Factorisation by iteration (RISCv):

```
.data
test_input: .word 4 3 7 8      #data to be factorised

.text
main:
    add t0, x0, x0              #t0 = 0, it represents loop counter [i].
    addi t1, x0, 4              #t1 = 4, Number of elements in array setting the loop
                                #limit.
    la t2, test_input           #t2 address is pointing to the address at test_input.
main_loop:
    beq t0, t1, main_exit       #When t0 == t1, leave the loop (After the final element, exit the loop).
    slli t3, t0, 2              #Multiplying t0 by 4 since we want to get the byte offset. slli is
                                #multiplication by 2^n.
    add t4, t2, t3              #t4 = base address + base offset. Address of test_input [i].
    lw a0, 0(t4)               #Load test_input[i] into a0 argument register (to be inputted to the factorial
                                #function).

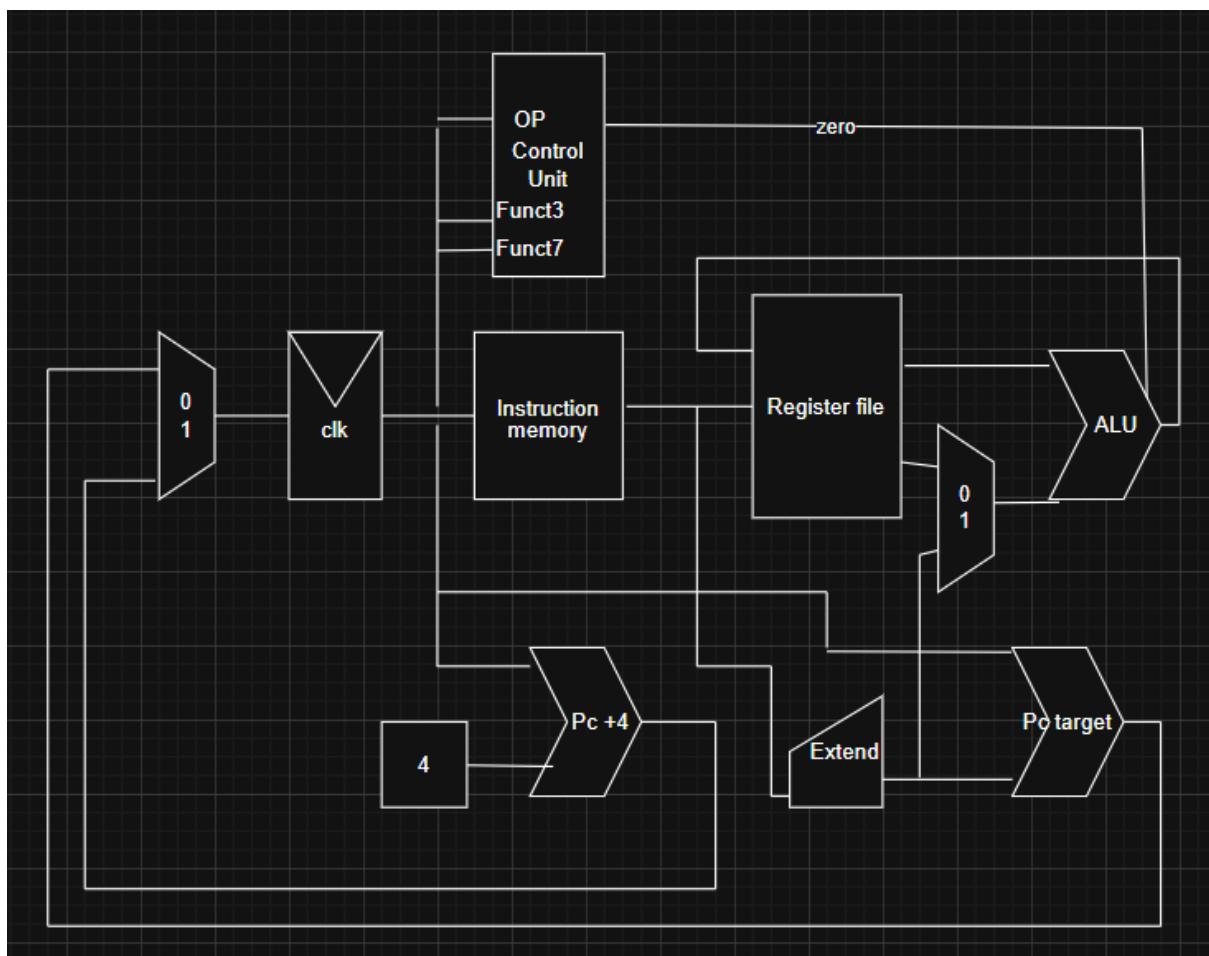
    addi sp, sp, -20            #Allocate stack space (-20 for 5 registers (5*4)).
    sw t0, 0(sp)                #Write t0(loop counter) into memory.
    sw t1, 4(sp)                #Write t1(loop limit) into memory.
    sw t2, 8(sp)                #Write t2(pointer to base address of array) into memory.
    sw t3, 12(sp)               #Write t3(byte offset) into memory.
    sw t4, 16(sp)              #Write t4(Address of test_input [i]) into memory.
```


jal ra, factorial	#Call the factorial function and return address to be in ra.
lw t0, 0(sp)	#Restoring all the registers from stack.
lw t1, 4(sp)	
lw t2, 8(sp)	
lw t3, 12(sp)	
lw t4, 16(sp)	
addi sp, sp, 20	#Deallocate stack space.
addi a1, a0, 0	#Move a0 to a1 to print.
addi a0, x0, 1	#a0 = 1 for ecall code to print integer.
ecall # Print Result	#Execute the instruction.
addi a1, x0, ' '	#a1 = ASCII for space character.
addi a0, x0, 11	#a0 = 11 for ecall code to print character.
ecall	#Execute code for print character.
addi t0, t0, 1	#t0++, increment loop counter.
jal x0, main_loop	#Jump to the main_loop.
main_exit:	
addi a0, x0, 10	#a0 = 10, Ecall code for exit.
ecall # Exit	#Execute instruction to exit programme.
factorial:	#t0 not needed in this factorial function since we do not have a loop counter.
addi t1, x0, 1	#t1 = 1, t1 represents i. Starting at 1.
addi t2, x0, 1	#t2 = 1, Initialised to 1.
factorial_loop:	
bgt t1, a0, factorial_exit	#if t1(i) > a0(n), branch to factorial_exit.
	#Multiplication of t2 * t1 through addition.
add t3, x0, x0	#t3 = 0 (running total).
add t4, x0, t1	#t4 = (amount of times we do repeat addition).
add t5, x0, t2	#t5 = result to be added repeatedly.
multiplication:	
beq t4, x0, multiplication_exit	#If t4==0 branch to exit.
add t3, t3, t5	#equivalent to t3 += result.
addi t4, t4, -1	#decrementing t4.
jal x0, multiplication	#loop again until exit condition.
multiplication_exit:	
add t2, t3, x0	#result = product.
addi t1, t1, 1	#i++.
jal x0, factorial_loop	#branch to factorial_loop.
factorial_exit:	
add a0, t2, x0	#t2 to the argument register for returning.
jr ra	#return result.

When comparing both methods, it seems that factorisation by iteration is faster because it doesn't have the need to load and store memory into the stack after each element in the array therefore less instructions, so it is faster.

Task 2B

The design for the processor is for the factorisation by iteration, this has a repetitive addition used for its multiplication which can be optimised better in processor design. This is because it can be replaced with a mul instruction with fewer instructions so fewer cycles to speed up execution. Before there would be many cycles per iteration but with the new instruction it will only be 1 cycle per iteration.



Task 3

MNK loop

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```

void matrix_multiply(int m, int n, int k, double *A, double *B, double *C) {

    for (int i = 0; i < m; i++) {

        for (int j = 0; j < n; j++) {

            C[i*n + j] = 0;

            for (int l = 0; l < k; l++) {

                C[i*n + j] += A[i*k + l] * B[l*n + j];

            }

        }

    }

}

void create_matrix(int rows, int cols, double *matrix) {

    for (int i = 0; i < rows*cols; i++) {

        matrix[i] = (double)rand() / RAND_MAX;           //Initialisation of matrices A and B

    }

}

void show_matrix(char *name, int rows, int cols, double *matrix) {

    printf("\n%s (%dx%d):\n", name, rows, cols);           //Matrix display


                                                                //Show partial matrix preview

    int show_rows = (rows > 10) ? 10 : rows;

    int show_cols = (cols > 10) ? 10 : cols;

    for(int i = 0; i < show_rows; i++) {

        for(int j = 0; j < show_cols; j++) {

            printf("%6.2f ", matrix[i*cols + j]);

        }

        if(cols > show_cols) printf("...");

        printf("\n");

    }

}

```

```

    if(rows > show_rows) printf("... (%d more rows)\n", rows - show_rows);
}

int main() {

    int m, n, k;

                                //m = rows of A and C

                                //n = columns of B and C

                                //k = columns of A and rows of B

    srand(time(NULL)); //random number generator

                                //Matrix parameter input

    printf("Enter matrix sizes (m n k): ");
    scanf("%d %d %d", &m, &n, &k);

                                //Memory allocation for matrices

    double *A = malloc(m * k * sizeof(double));
    double *B = malloc(k * n * sizeof(double));
    double *C = malloc(m * n * sizeof(double));

                                //Initialize matrices A and B

    create_matrix(m, k, A);
    create_matrix(k, n, B);

                                //Execution time clock

    clock_t start = clock();
    matrix_multiply(m, n, k, A, B, C);
    clock_t end = clock();

                                //Print time

    printf("\nCalculation time: %.4f seconds\n", (double)(end - start)/CLOCKS_PER_SEC);

                                //Print matrices

    show_matrix("Matrix A", m, k, A);
    show_matrix("Matrix B", k, n, B);

```

```

show_matrix("Result Matrix C", m, n, C);

return 0;
}

```

When we increase the m, n and k parameters, the execution time increases. The execution time is not linear as the further we increase the parameters, the even longer the execution time. For example the m,n,k = 4 has negligible difference with m,n,k = 8 for execution time but m,n,k = 500 (0.6s) to m,n,k = 1000 (6s) has a 5 second difference.

MKN loop

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void matrix_multiply(int m, int n, int k, double *A, double *B, double *C) {
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            C[i*n + j] = 0.0; //First row initialised
        }

        // New i-l-j loop
        for (int l = 0; l < k; l++) {
            double a_val = A[i*k + l]; // Cache A value for this l
            for (int j = 0; j < n; j++) {
                C[i*n + j] += a_val * B[l*n + j];
            }
        }
    }
}

void create_matrix(int rows, int cols, double *matrix) {
    for (int i = 0; i < rows*cols; i++) {
        matrix[i] = (double)rand() / RAND_MAX;
    }
}

```

```

    }
}

void show_matrix(char *name, int rows, int cols, double *matrix) {

    printf("\n%s (%dx%d):\n", name, rows, cols);

    int show_rows = (rows > 10) ? 10 : rows;

    int show_cols = (cols > 10) ? 10 : cols;

    for(int i = 0; i < show_rows; i++) {

        for(int j = 0; j < show_cols; j++) {

            printf("%6.2f ", matrix[i*cols + j]);

        }

        if(cols > show_cols) printf("...");

        printf("\n");

    }

    if(rows > show_rows) printf("... (%d more rows)\n", rows - show_rows);

}

int main() {

    int m, n, k;

    srand(time(NULL));

    printf("Enter matrix sizes (m n k): ");

    scanf("%d %d %d", &m, &n, &k);

    double *A = malloc(m * k * sizeof(double));

    double *B = malloc(k * n * sizeof(double));

    double *C = malloc(m * n * sizeof(double));

    create_matrix(m, k, A);

    create_matrix(k, n, B);

    clock_t start = clock();

    matrix_multiply(m, n, k, A, B, C);

```

```

clock_t end = clock();

printf("\nCalculation time: %.8f seconds\n", (double)(end - start)/CLOCKS_PER_SEC);

show_matrix("Matrix A", m, k, A);
show_matrix("Matrix B", k, n, B);
show_matrix("Result Matrix C", m, n, C);

free(A);
free(B);
free(C);

return 0;
}

```

There is now a different order of initialisation for the nested loops. When we input m,n,k = 1000 we get 3.04 seconds which is double as quick as previous loop configuration.

The difference in performance is because of memory access patterns and cache behaviour. The MKN loop is optimised because the memory is accessed contiguously, the spatial locality is optimised so each adjacent element is used immediately so cache lines are utilised effectively. The i element in the A matrix ($a_val = A[i*k + l]$) is reused by storing the value in a register so no need for reloading for the j values. It works by accessing rows instead of columns which is the order that matrices are stored. The inefficient MNK loop accesses in columns meaning that it only needs 1 value per cache line leading to strided access where it skips elements in the row, wasting the cache line. $C[i][j]$ has temporal locality as it updates every iteration so to update it every iteration without loading it from memory every time, it stays in the cache.

[1]

C. Babbage, *Passages from the life of a philosopher*. Westmead, Farnborough, England: Gregg International Publishers, 1969.

[2]

J. A. Fleming, "Instrument for converting alternating electric currents into continuous currents," U.S. Patent 803,684, Nov. 7, 1905.

[3]

G. E. Moore, "Cramming more components onto integrated circuits," *Electronics*, vol. 38, no. 8, Apr. 1965

[4]

N. Rinaldi and M. Schroter, *Silicon-germanium heterojunction bipolar transistors for mm-wave systems : technology, modeling and circuit applications*, 1st ed.. 2018.

[5]

A. Morrison and Y. Afek, "Fast concurrent queues for x86 processors," *Proc. 18th ACM SIGPLAN Symp. Princ. Pract. Parallel Program*

[6]

M. Horowitz, E. Alon, D. Patil, S. Naffziger, R. Kumar, and K. Bernstein, "Scaling, power, and the future of CMOS," in *Proceedings of the IEEE International Electron Devices Meeting (IEDM)*, Dec. 2005

[7]

D. Hisamoto, T. Kaga, Y. Kawamoto, and E. Takeda, "A fully depleted lean-channel transistor (DELTA)—A novel vertical ultra thin SOI MOSFET," in *International Technical Digest on Electron Devices Meeting (IEDM)*, Washington, DC, USA, 1989

[8]

S. Singh, S. K. Sharma, D. Nandan, and SpringerLink, *Beyond Si-Based CMOS Devices Materials to Architecture*, 1st ed. 2024.. 2024.

[9]

D. J. Griffiths and D. F. Schroeter, *Introduction to quantum mechanics*, Third edition.. 2018.

[10]

M. Wand, K. Malinová, J. Schmidhuber, I. V. Tetko, and SpringerLink, *Artificial Neural Networks and Machine Learning – ICANN 2024 33rd International Conference on Artificial Neural Networks, Lugano, Switzerland, September 17–20, 2024, Proceedings, Part VII*, 1st ed. 2024.. 2024.